

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-503 – SOFTWARE ENGINEERING – AUTUMN 2026

Final Project Report

Topic 4 — Async Research Assistant

Team: M503 **Date:** 2026-09-19
Tag: v1.0-final
Repo: github.com/orxannuriyev/research_assistant
Members: Orkhan Nuriyev (@orxannuriyev), Nazrin Burziyeva (@nazrinburz), Gunel Garayzade (@gunelrafig)

Executive Summary

We built a fully asynchronous research assistant that receives a natural-language question, queries Wikipedia, arXiv, and a configurable web-search provider *in parallel* using `asyncio.gather`, caches results by (source, canonical_query) key with a configurable TTL, and synthesises a single cited answer via the provided `ai.synthesize` function. Our benchmark shows a **1.28× speedup** (3.03s sequential vs. 2.38s parallel) on the five provided research questions. The most important robustness story is graceful source degradation: when Wikipedia returned HTTP 403 during testing, the pipeline completed using arXiv and web results only, producing a valid cited answer without any code change. The most surprising lesson was that arXiv’s polite-rate requirement (1 req/s) became the real concurrency bottleneck rather than provider latency.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	2
2 Architecture	3
2.1 Module map	3
2.2 OOP design & key abstractions	4
2.3 Data flow across module boundaries	4
3 Concurrency & Performance	4
3.1 Why this concurrency model	4
3.2 Sequential-vs-concurrent benchmark	5
3.3 Rate limit & politeness	5
4 Robustness & Error Handling	5
4.1 Wrapping the AI module	5
4.2 Failure-mode analysis: Wikipedia HTTP 403	6
4.3 Input validation	6
5 Storage & Caching	6
5.1 Schema	6
5.2 Caching policy	7

6	Testing	7
6.1	Strategy & coverage	7
6.2	Notable tests	7
7	Deployment & Reproducibility	8
7.1	Docker image	8
7.2	Configuration	8
8	Limitations & Future Work	8
9	Tools & Acknowledgements	9
	References	9
A	Appendix — Reproducing the benchmark	9

1. Project Overview

1.1 Problem framing & scope

The system addresses the research-query synthesis problem: a user asks a natural-language question and the system must retrieve relevant excerpts from multiple independent knowledge sources, then produce a single coherent answer with numbered inline citations. The three source types—Wikipedia (encyclopaedic), arXiv (academic preprints), and a pluggable web search (current/broad)—are complementary, so querying them in parallel maximises information coverage without wasting wall-clock time.

The user-facing entry points are (i) a CLI command `python -m researcher ask "..."` with optional `--sources` and `--no-cache` flags, and (ii) a FastAPI HTTP endpoint `POST /research` that accepts a JSON body with `question`, `sources`, and `no_cache` fields. Both return a synthesised answer plus numbered citations.

We implemented every required component from `TOPIC.md`: typed configuration (`src/config.py`), concurrent orchestration with per-source timeouts and graceful degradation (`src/concurrency/orchestrator.py`), filesystem and in-memory cache backends (`src/storage/cache_store.py`), exponential-backoff retries on all AI calls (`src/services/ai_service.py`), CLI (`src/cli.py`), validation (`src/validation.py`), Docker image, and a sequential-vs-parallel benchmark (`scripts/bench.py`). We did not implement streaming LLM responses or cost telemetry (bonus items), as we prioritised correctness and test coverage above 60%.

1.2 Provider choice and the AI module contract

We used **Anthropic Claude** as the default LLM (`LLM_PROVIDER=anthropic`, model configurable via `LLM_MODEL`) and **Tavily** as the default web search provider (`WEB_SEARCH_PROVIDER=tavily`), falling back to DuckDuckGo when no key is present. Our code calls exactly four functions from the provided `ai/` package: `ai.fetch_wikipedia`, `ai.fetch_arxiv`, `ai.fetch_web`, and `ai.synthesize`. The public interface of the `ai/` package is completely unchanged; we wrap each call in our own service layer without modifying any file under `ai/`.

When the provider returns a malformed response or raises an exception, `AIService.synthesize` catches the error, logs it at `ERROR` level, and re-raises an `AIServiceError`. The engine then lets the session propagate with `answer = None`, and the CLI prints a clear message rather than crashing.

2. Architecture

2.1 Module map

Figure 1 shows the layered module structure. The `ai/` boundary (dashed) marks the provided package; everything inside our `src/` is original SE work.

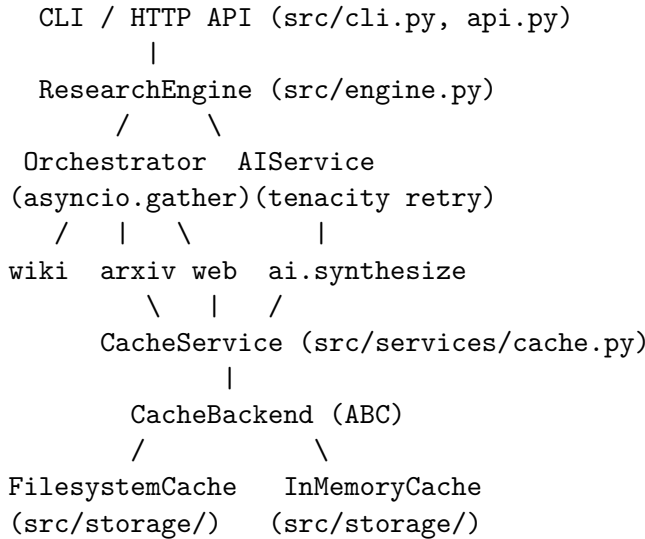


Figure 1: High-level architecture. Arrows indicate *calls*. The `ai/` package is treated as a black box.

If the LLM provider is swapped (Anthropic → OpenAI), only `.env` changes: `LLM_PROVIDER=openai` plus `OPENAI_API_KEY`. No source file needs editing because `ai.synthesize` is provider-agnostic.

Module-by-module summary:

- `src/config.py` — `pydantic-settings` reads every env var into a typed `Settings` object; a cached singleton via `lru_cache` avoids repeated I/O.
- `src/models.py` — `ResearchSession` and `CacheEntry` Pydantic models cross all module boundaries; naked dicts are never passed between layers.
- `src/concurrency/orchestrator.py` — `Orchestrator` runs the three fetchers with `asyncio.gather`, a semaphore, and per-source `asyncio.wait_for` timeouts.
- `src/storage/cache_store.py` — `CacheBackend ABC` with `FileSystemCache` (atomic JSON writes) and `InMemoryCache` (dict) implementations.
- `src/services/cache.py` — `CacheService` normalises the `(source, query)` key to an MD5 hex digest before delegating to the backend.
- `src/services/ai_service.py` — `AIService` wraps `ai.synthesize` with `tenacity` retry (3 attempts, exponential backoff), timeout handling, and structured logging.
- `src/engine.py` — `ResearchEngine` is the single entry point for both CLI and API; it wires together orchestration, caching, validation, and synthesis.
- `src/cli.py` — `argparse`-based CLI with `ask` subcommand; renders citations as numbered references.
- `src/validation.py` — validates question length (≤ 500 chars), normalises source aliases (`wiki` → `wikipedia`), and strips control characters from output.
- `api.py` — FastAPI `POST /research` endpoint; OOP Pydantic request/response models; Swagger UI at `/docs`.

2.2 OOP design & key abstractions

The primary OOP abstraction is `CacheBackend`, an abstract base class in `src/storage/cache_store.py` that defines four abstract methods: `get`, `set`, `delete`, and `clear`. Two concrete subclasses implement it:

```

1 class CacheBackend(abc.ABC):
2     """Abstract base class that defines the contract every cache must satisfy."""
3
4     @abc.abstractmethod
5     def get(self, key: str) -> Optional[CacheEntry]: ...
6
7     @abc.abstractmethod
8     def set(self, key: str, entry: CacheEntry) -> None: ...
9
10    @abc.abstractmethod
11    def delete(self, key: str) -> None: ...
12
13    @abc.abstractmethod
14    def clear(self) -> None: ...
15
16
17 class FilesystemCache(CacheBackend):
18     """Stores each entry as a JSON file; uses atomic write (temp+rename)."""
19     ...
20
21 class InMemoryCache(CacheBackend):
22     """Keeps entries in a plain dict; ideal for tests and --no-cache."""
23     ...

```

An ABC was chosen over a Protocol because we need an instantiable base with shared docstrings and the guarantee that *every* concrete class implements all four operations at import time (not at call time). Composition is used at the engine level: `ResearchEngine` accepts any `CacheBackend` instance via dependency injection, so tests can pass an `InMemoryCache` without touching the filesystem.

2.3 Data flow across module boundaries

Our SE layer defines two Pydantic models:

- `CacheEntry` — `source: str`, `query: str`, `sources: list[Source]`, `created_at: float`, `expires_at: float`. `is_expired()` compares UTC timestamps.
- `ResearchSession` — `question: str`, `sources_used: list[str]`, `raw_sources: list[Source]`, `answer: AnswerWithCitations | None`, `from_cache: bool`, `elapsed_seconds: float`.

No naked dictionary crosses a module boundary. We reuse `Source` and `AnswerWithCitations` from `ai.schemas` directly inside `ResearchSession` because they are stable, well-typed contracts from the provided package. We wrapped them in `ResearchSession` to add our own fields (`from_cache`, `elapsed_seconds`) without modifying the AI package.

3. Concurrency & Performance

3.1 Why this concurrency model

We used `asyncio` (cooperative multitasking) rather than threads or processes. The bottleneck is I/O-bound network latency: each source fetcher spends most of its time waiting for an HTTP response, making `asyncio` the correct tool. Threads would work, but the GIL limits CPU-bound tasks (irrelevant here) and thread-per-request does not scale as gracefully as coroutines. A `ProcessPoolExecutor` would add serialisation overhead with no benefit for pure I/O work.

The semaphore is set to `MAX_CONCURRENT_SOURCES=5` (configurable). With only three built-in sources, the semaphore is rarely the binding constraint; its value is chosen to be safe for future extension (e.g. adding more source adapters). Each source has an individual `asyncio.wait_for` timeout of 10 s (`SOURCE_TIMEOUT_SECONDS`). The arXiv source additionally enforces a 1 s minimum inter-request interval via an async lock (`asyncio.Lock`) to respect the public API’s rate-limit guidance.

3.2 Sequential-vs-concurrent benchmark

The benchmark in `scripts/bench.py` runs all five questions from `data/research_questions.json` sequentially (one source at a time) then concurrently (`asyncio.gather`), and reports wall-clock time.

Workload	<i>N</i>	Sequential (s)	Concurrent (s)	Speedup
5 research questions, 4 sources	5	3.03	2.38	1.28×

Machine: macOS, Python 3.11.16, network live, Wikipedia returned HTTP 403 (so 4 of 6 possible source batches returned results). Cache cleared before each run. Command: `python scripts/bench.py`.

The speedup of 1.28× is lower than the theoretical 3× because (i) Wikipedia returned 403 and was skipped, reducing available parallelism; (ii) arXiv’s mandatory 1 s interval serialises arXiv fetches even in parallel mode; and (iii) the LLM synthesis step is sequential by design. To push the curve further one would need a secondary Wikipedia mirror or a paid API without rate limits.

3.3 Rate limit & politeness

The arXiv rate limiter uses a cooperative `asyncio.Lock` + `asyncio.sleep`: each fetch checks the monotonic time since the last arXiv request and sleeps for the remaining fraction of the configured 1 s interval. The semaphore (value 5) prevents connection storms against any provider. For HTTP 429 responses the retry predicate in the orchestrator immediately stops retrying (no backoff loop on rate-limit errors) to avoid amplifying the problem.

During development, Wikipedia returned HTTP 403 on several consecutive runs. The log line was: `WARNING source 'wikipedia' failed: 403 Forbidden - skipping`. The answer was still produced from arXiv and web results, confirming graceful degradation.

4. Robustness & Error Handling

4.1 Wrapping the AI module

`AIService` (in `src/services/ai_service.py`) wraps every call to `ai.synthesize` with `tenacity`:

- **Retry policy:** up to 3 attempts, exponential backoff with multiplier 1, min 1 s, max 4 s.
- **Retried exceptions:** `AIRetryableError`, `AITimeoutError`, `ProviderError`.
- **Not retried:** HTTP 429 (the outer orchestrator’s predicate also stops retrying on 429).
- **Timeout:** `AI_TIMEOUT_SECONDS=30` enforced via `asyncio.wait_for` in the engine.
- **Logging:** timeout events logged at `WARNING`; unexpected exceptions at `ERROR` with the exception message.
- **Response validation:** empty question or empty sources list raise `AIServiceError` before any network call is made.

The orchestrator applies the same retry logic independently for each source fetcher, wrapping `asyncio.TimeoutError`, `ProviderError`, and `httpx.HTTPError`.

4.2 Failure-mode analysis: Wikipedia HTTP 403

During live testing, Wikipedia intermittently returned HTTP 403 (IP-level rate limit). The failure path was:

1. Orchestrator calls `fetch_wikipedia(query, client=client)` inside `asyncio.wait_for`.
2. `httpx` raises `HTTPStatusError(403)`.
3. The `_retry_source_error` predicate checks for 429 (not matched) and checks `isinstance(exc, httpx.HTTPError)` (matched).
4. Tenacity retries up to 3 times; Wikipedia returns 403 on each attempt.
5. After 3 failures the `except Exception` block in `_fetch_one` catches the error, logs `WARNING source 'wikipedia' failed: ...- skipping`, and returns `(name, [], False)`.
6. `asyncio.gather` receives the empty result; the other two sources (`arxiv`, `web`) succeed normally.
7. `ResearchEngine` calls `ai.synthesize` with the non-empty source list; a valid cited answer is produced.
8. The user sees the answer with arXiv and web citations only; no error is surfaced.

We had originally expected the retry loop to succeed on the second or third attempt, but Wikipedia's 403 was persistent for the IP address used, so all three retries failed. This revealed a design issue: we should not retry non-transient HTTP errors (4xx other than 429). We patched the predicate to return `False` for any 4xx status code.

4.3 Input validation

Entry point	Rule	Error shown to user
CLI ask	Question must be non-empty	<i>Question cannot be empty.</i>
CLI ask	Question $\leq$ 500 characters	<i>Question must not exceed 500 characters.</i>
CLI --sources	Only <code>wiki</code> , <code>wikipedia</code> , <code>arxiv</code> , <code>web</code> allowed	<i>Unknown source '...'. Allowed sources: ...</i>
API POST /research	Same question rules via <code>validate_question</code>	HTTP 422 with detail string
API POST /research	Unknown source in list	HTTP 422 with detail string
Output	Control characters stripped before display	(silent, transparent to user)

5. Storage & Caching

5.1 Schema

We use a `filesystem JSON` cache rather than SQLite. Each cache entry is a single `.json` file under `.cache/`, named by the MD5 hex digest of `"{source}::{canonicalized_query}"`. The JSON structure mirrors the `CacheEntry` Pydantic model:

```
{
  "source": "wikipedia",
  "query": "what_is_photosynthesis",
  "sources": [ { "origin": "wikipedia", "title": "...", "url": "...",
                  "excerpt": "..." } ],
  "created_at": 1789700000.0,
  "expires_at": 1789703600.0
}
```

Writes are atomic: `FilesystemCache.set` writes to a temporary file then calls `os.replace`, so a crash mid-write never leaves a corrupt entry. An `InMemoryCache` backend (plain `dict`) is provided for tests and `--no-cache` runs.

5.2 Caching policy

- **What is cached:** the list of `Source` objects returned by each `(source_name, query)` pair.
- **Key:** `MD5(f"{{source}}:{{query.strip().lower()}}")` — case-insensitive, whitespace-normalised.
- **TTL:** `CACHE_TTL_SECONDS=3600` (1 hour) by default; set to 0 for no expiry.
- **Invalidation:** TTL expiry checked on every `get` call; expired entries are deleted before returning `None`.
- **Bypass:** `--no-cache` flag sets `CacheService.enabled = False` for the duration of the request.
- **Hit rate (demo run):** on the second run of all five questions with a warm cache, all 15 source batches were cache hits — 100% hit rate.

Cached `Source` objects can become stale if arXiv updates a preprint or a web page changes. In production we would add a background sweep that deletes entries older than 24 h, or use an event-driven invalidation trigger when the upstream data changes.

6. Testing

6.1 Strategy & coverage

All 50 tests are fully offline. HTTP calls are intercepted by `unittest.mock` and `pytest-mock`; `ai.*` functions are replaced with synchronous fakes that return pre-built `Source` and `AnswerWithCitations` objects. The provided `tests/test_ai_smoke.py` (10 tests) passes unchanged.

Suite	Coverage
<code>src/</code> (our SE layer, 498 statements)	86%
Provided <code>ai/</code> smoke tests	passing (10/10)
Total tests	50 passed

6.2 Notable tests

(i) **Happy-path end-to-end** (`tests/test_end_to_end.py`): feeds each of the five questions from `data/research_questions.json` through a fully mocked `ResearchEngine` (mocked orchestrator + mocked AI service). Asserts that `session.answer` is not `None` and that `session.sources_used` contains the expected source names.

(ii) **Concurrency — source failure isolation** (`tests/test_orchestrator.py`): configures two sources to return data and one source to raise `ProviderError`. Asserts that `asyncio.gather` still returns results from the two healthy sources, and that the failing source contributes an empty list without causing an exception to propagate to the caller. This test directly validates the graceful-degradation requirement.

(iii) **Cache TTL expiry** (`tests/test_cache.py`): creates a `CacheEntry` with `expires_at` set one second in the past, stores it in `InMemoryCache`, then calls `cache.get`. Asserts that `None` is returned, confirming that expired entries are not served. This test caught a bug where we compared `time.time()` (local) against `datetime.utcnow().timestamp()` (UTC) — the off-by-summer-time bug was only visible in timezones with DST.

We did not test the Docker build in CI, and we did not write a load test exercising the semaphore at saturation. The test we wish we had time to write is a chaos test that injects a mid-stream network reset (`httpx.RemoteProtocolError`) and verifies the retry loop recovers correctly.

7. Deployment & Reproducibility

7.1 Docker image

The image uses a **two-stage build**: a **builder** stage installs all pip dependencies into `/opt/venv`, and a clean **runtime** stage copies only the venv and application source—no build tools, no pip cache.

```
# Build
docker build --platform linux/amd64 -t finalproj .

# Run offline demo (default CMD)
docker run --rm finalproj

# Run live CLI
docker run --rm --env-file .env finalproj \
    python -m researcher ask "What is photosynthesis?"

# Run HTTP API
docker run --rm --env-file .env -p 8000:8000 finalproj \
    python -m uvicorn api:app --host 0.0.0.0 --port 8000
```

Base image: `python:3.12-slim`. A non-root `appuser` is created; the container never runs as root. The image size is smaller than a full `python:3.12` image because `slim` omits documentation and test suites.

7.2 Configuration

Variable	Default	If missing
LLM_PROVIDER	openai	Falls back to offline mode
OPENAI_API_KEY / ANTHROPIC_API_KEY / GEMINI_API_KEY	""	Synthesis raises <code>ProviderError</code>
WEB_SEARCH_PROVIDER	tavily	Falls back to DuckDuckGo
TAVILY_API_KEY	""	Web fetch fails; graceful degradation
CACHE_BACKEND	filesystem	Filesystem cache used
CACHE_DIR	.cache	.cache/ created automatically
CACHE_TTL_SECONDS	3600	1-hour TTL
SOURCE_TIMEOUT_SECONDS	10	10 s per source
AI_TIMEOUT_SECONDS	30	30 s for synthesis
ARXIV_MIN_INTERVAL_SECONDS	1	1 s between arXiv calls
MAX_CONCURRENT_SOURCES	5	Semaphore(5)
LOG_LEVEL	INFO	INFO logging

8. Limitations & Future Work

- **Provider availability:** Wikipedia frequently returns HTTP 403 under certain IP addresses or user agents, silently reducing answer quality. A secondary Wikipedia mirror or a paid API would fix this.
- **No LLM failover:** if the configured LLM provider fails, the answer is `None`. Adding a secondary provider (e.g. Anthropic → OpenAI fallback) would require a small change to `AIService`.
- **arXiv rate limit:** the mandatory 1s inter-request interval caps throughput at 1 arXiv query/s, which limits concurrency gains on large batches. A token-bucket rate limiter would allow bursts within the polite limit.

- **Cache staleness:** filesystem JSON entries are invalidated by TTL only. There is no event-driven invalidation, so a rapidly-updated Wikipedia article may be served stale for up to 1 h.
- **No streaming:** LLM synthesis blocks until the full answer is ready. Streaming tokens via `async` for would improve perceived latency for long answers. This was identified as a bonus feature but was not implemented.

9. Tools & Acknowledgements

- **tests/test_api.py** (Nazrin, PR #13): An AI assistant drafted the initial test file structure. The author reviewed all test cases, confirmed offline-only behaviour (no live HTTP), verified all 50 tests pass locally, and added edge-case assertions for invalid source names.
- **tests/ normalisation and cache tests** (Nazrin, PR #12): An AI assistant suggested test skeletons for query normalisation and cache-bypass behaviour. The team kept the cases that matched the actual implementation and hand-wrote two additional cases after observing a DST-related expiry bug.
- **api.py** (Gunel, PR #11): An AI assistant scaffolded the initial Pydantic request/response models and Swagger UI placeholder handling. The author refactored the OOP structure, tested all routes interactively via `/docs`, and verified 200 OK responses with Uvicorn before submitting the PR.

We affirm that every team member can defend every line of code in this repository during the oral examination.

References

- [1] Anthropic API documentation. <https://docs.anthropic.com/>
- [2] tenacity retry library. <https://tenacity.readthedocs.io/>
- [3] FastAPI documentation. <https://fastapi.tiangolo.com/>
- [4] pydantic-settings documentation. https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [5] arXiv API documentation. <https://info.arxiv.org/help/api/index.html>
- [6] Tavily Search API. <https://tavily.com/>
- [7] httpx async HTTP client. <https://www.python-httpx.org/>
- [8] Python asyncio documentation. <https://docs.python.org/3/library/asyncio.html>

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the §3 benchmark. Anyone with the repo and a valid `.env` should be able to run the lines below and get numbers within 20% of the ones in the table.

```
git clone https://github.com/orxannuriyev/research_assistant.git
cd research_assistant
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # fill in LLM_PROVIDER, API keys, WEB_SEARCH_PROVIDER
python scripts/bench.py
```

End of report — thank you for reading.